Reading a plan: the short version

Four pages: the rules, then one real plan answered the way the exam grades

Advanced Data Management

2026-11-01

1 The lines and what they mean

Concept: Node line

`(cost=start..total rows=est width=w)`: the planner's *belief*, in abstract units (not ms). `start` = work before the first row leaves the node (for a breaker: its whole input). `rows` = estimated rows *per execution*.
`(actual time=first..last rows=act loops=n)`: measured, *per loop*. A node on the inner side of a Nested Loop runs once per outer row, so: total rows = `act` × `loops`; time = `last` × `loops`. `first..last` is first row and last row, not a duration.

Concept: Sub-lines

- `Filter` + `Rows Removed by Filter`: the predicate was tested on each row *after* reading it; the removed count is wasted work.
- `Index Cond`: answered inside the index, no row read for it.
- `Recheck Cond` + `Heap Blocks: exact=N`: a bitmap scan; N is the number of heap pages it visited.
- `Heap Fetches`: index-only scan; 0 means no heap page at all.
- `Hash Cond` / `Merge Cond` / `Join Filter`: the join condition (the Join Filter is applied after pairing).
- `Buckets ...` `Batches ...` `Memory Usage`: the hash build; `Batches` > 1 means it spilled to disk.
- `Sort Key`, `Sort Method: quicksort Memory` (fit) / `external merge Disk` (spilled) / `top-N heapsort` (ORDER BY + LIMIT).
- `Group Key`: the GROUP BY columns.
- `Buffers: shared hit=H read=R`: pages found in `shared_buffers` / not found there (fetched from the OS cache *or* disk; `read` $\neq$ disk). Per node, children included. `temp read/written` = a spill.

Concept: The tree

Indentation gives the children. For a join, the first child is the **outer** (nested loop) or **probe** (hash join) side, the second the **inner** or **build** side (the one under `Hash`). Read bottom-up; execution starts at the leftmost leaf of the deepest breaker.
Blocking (consumes all input before its first output): Sort, Hash (build), HashAggregate, Materialize, Bitmap Index Scan. **Streaming**: Seq Scan, Index Scan, Bitmap Heap Scan, Filter, Limit, Nested Loop, Merge Join, hash probe, GroupAggregate on sorted input. A LIMIT above a breaker

saves nothing below it.
Reconstruct the SQL: Scan on `t a` = FROM; `Hash/Merge Cond` = ON; `Filter/Index Cond` = WHERE; `Group Key` = GROUP BY; `Sort Key` = ORDER BY; `Limit` = LIMIT.

Technique: The five moves, in the order you write them

1. **Leaves**: for each table, scan type, filter, estimated vs actual rows.
2. **Up**: join algorithm and order, aggregate, sort.
3. **Cause**: the *lowest* node with a large q-error ($\max(\text{est}/\text{act}, \text{act}/\text{est}) \geq 10$); everything above inherits it. An under-estimate is the dangerous direction: it buys per-row plans (nested loop, index lookups) for a big input.
4. **Resources**: `Buffers` per node (the `read` count, not a ratio), `Sort Method`, `Batches`, `loops` on the inner side, `Heap Blocks`. Seconds $\approx$ `read` $\times$ random price.
5. **Decide**: wrong plan from a wrong estimate $\rightarrow$ statistics fix; right plan but slow $\rightarrow$ memory, index, cache; name the command.

Concept: Where an estimate comes from, and the three fixes

Scan `rows=` = (MCV frequency, else $1/n_{\text{distinct}}$, else histogram share) $\times N$; each extra predicate multiplies by its own selectivity (independence); join rows by the join formula.
Stale (`n_live_tup` $\neq$ `reltuples`, old `last_analyze`) $\rightarrow$ `ANALYZE t;`
Correlated columns (each frequency right, the product wrong; ANALYZE does not help) $\rightarrow$ `CREATE STATISTICS s (dependencies) ON a, b FROM t; ANALYZE t;`
Skew (a rare value estimated at $1/d$) $\rightarrow$ `ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000; ANALYZE t;`

Concept: What each fix changes

Extended statistics: estimate right $\rightarrow$ join algorithm and order change. **Composite index on the filter**: Filter moves into Index Cond; pages unchanged if rows are scattered. **CLUSTER**: correlation $\rightarrow$ 1, Heap Blocks collapse, the join untouched; a rewrite under lock. **Covering index**: index-only, Heap Fetches 0. **work_mem**: Disk/Batches disappear. **Hash join instead of nested loop**: build on the smaller *filtered* input (check rows $\times$ width against work_mem), probe with a Seq Scan: sequential pages instead of loops of random ones.

2 One real plan, five answers

`bookly`: `orders` 3,000,000 rows, 28,038 pages, 107 rows/page; `customers` 300,000 rows, 2,500 pages, `customers_pkey` 3 levels. `pg_stats`: `status` MCV `returned` 0.05; `country` MCV `FR` 0.20, both fresh. A random read costs about 60 μs here (part OS cache, part SSD). `work_mem` 4 MB.

```
SELECT c.name, count(*) AS n, sum(o.amount) AS refunded
FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE o.status = 'returned' AND o.country = 'FR'
GROUP BY c.name ORDER BY refunded DESC LIMIT 20;
```

```
Limit  (cost=115982.1..115982.2 rows=20)
       (actual time=3480.2..3480.3 rows=20 loops=1)
  Buffers: shared hit=458038 read=50000
  ->  Sort  (cost=115982.1..116057.1 rows=30000)
            (actual time=3480.1..3480.2 rows=20 loops=1)
        Sort Key: (sum(o.amount)) DESC
        Sort Method: top-N heapsort  Memory: 27kB
        ->  HashAggregate  (cost=114632.1..114932.1 rows=30000)
                  (actual time=3440.6..3462.3 rows=98210 loops=1)
              Group Key: c.name
              ->  Nested Loop  (cost=0.42..114482.1 rows=30000)
                        (actual time=0.3..3390.8 rows=120000 loops=1)
                    ->  Seq Scan on orders o
                          (cost=0.00..65538.0 rows=30000)
                          (actual time=0.1..410.5 rows=120000 loops=1)
                          Filter: ((status = 'returned') AND (country = 'FR'))
                          Rows Removed by Filter: 2880000
                          Buffers: shared hit=28038
                    ->  Index Scan using customers_pkey on customers c
                          (cost=0.42..1.63 rows=1)
                          (actual time=0.020..0.024 rows=1 loops=120000)
                          Index Cond: (id = o.customer_id)
                          Buffers: shared hit=430000 read=50000
Execution Time: 3482.0 ms
```

Worked example: Q1. Tree, blocking or streaming, why LIMIT does not help

Limit ← Sort ← HashAggregate ← Nested Loop {outer: Seq Scan `orders`; inner: Index Scan `customers`, once per outer row}. Seq Scan: streaming (filters row by row). Index Scan: streaming (one lookup per outer row). Nested Loop: streaming (emits each pair as the inner returns it). HashAggregate: **blocking** (a customer's sum is final only after its last order). Sort: **blocking** (the largest sum is known only when all groups are in). Limit: streaming.

Read it off the times: the join ends at 3,390.8 ms, the HashAggregate's first output is at 3,440.6 ms, the Sort's at 3,480.1 ms, the Limit's 20 rows at 3,480.3 ms. The LIMIT saved only the sort's output phase (`top-N heapsort`, 27 kB); 3.4 s were spent below two breakers that cannot stop early.

Worked example: Q2. Where do the estimates come from, q-error, the failure, the command

`rows=30000` on the Seq Scan: $0.05 \times 0.20 = 0.01$, times 3,000,000: two MCV frequencies multiplied under independence. Actual 120,000: q-error = 4, an *under*-estimate. French orders are returned four times more often than average: `status` and `country` are correlated. `ANALYZE` does not help (both single frequencies are already right). Fix:

```
CREATE STATISTICS s_ret (dependencies) ON status, country FROM orders;  ANALYZE orders;
```

New estimate $\approx$ 120,000, and at that count the planner abandons the nested loop (Q4).

Worked example: Q3. Where the time goes, and the Buffers line

Own times: Seq Scan 410 ms; inner lookups $0.024 \times 120{,}000 = 2{,}880$ ms; HashAggregate 72 ms; Sort 18 ms. The inner lookups own 83 %.
`hit=430000 read=50000`: 120,000 lookups × (3 index pages + 1 heap page) = 480,000. The index pages are the same hot pages every time (hits); the heap page differs per customer and 50,000 of those fetches missed. $50{,}000 \times 60\,\mu\text{s} = 3.0$ s ≈ the 2.9 s measured: the time is the random heap reads of `customers`.
Hit ratio 458,038/508,038 = 90 %: useless here; the cost is the 50,000 `reads` on one node. A bitmap scan on a `status` index would show `Heap Blocks` $= 28{,}038 \times (1 - 0.95^{107}) = 27{,}922$, the whole table: no gain.

Worked example: Q4. The plan the planner should have chosen

With 30,000 outer rows the lookups were priced $30{,}000 \times 1.63 \approx 49{,}000$; at the true 120,000 they are 195,600, so the plan as run is worth $65{,}538 + 195{,}600 \approx 261{,}000$ units (3.5 s). Hash join: build on the smaller *filtered* input, the 120,000 `orders` rows × 80 B = 9.6 MB > 4 MB (`Batches: 4`; `SET LOCAL work_mem = '16MB'` for one batch); probe with a Seq Scan of `customers`, $2{,}500 + 300{,}000 \times 0.01 = 5{,}500$. Total $\approx 65{,}538 + 5{,}500 + 0.0025 \times 420{,}000 \approx 72{,}000$ units, 3.6 times less; in time, 2,500 sequential pages (40 ms) instead of 50,000 random reads: about 0.5 s instead of 3.5 s.

Worked example: Q5. Four proposals, what each changes, which first

(i) `CREATE INDEX ON orders (status, country)`: Seq Scan becomes a Bitmap Heap Scan, `Heap Blocks` ≈ 18,400 (1 % of rows, scattered), estimate still 30,000, the 120,000 lookups remain: 3.3 s. (ii) `CLUSTER orders USING` that index: matches become $\lceil 120{,}000/107 \rceil = 1{,}122$ contiguous pages, the scan drops to ms, the join unchanged: 3.0 s, plus a 219 MB rewrite under lock that decays. (iii) extended statistics: estimate 120,000 → hash join: 0.5 s. (iv) covering index `(status, country) INCLUDE (customer_id, amount)`: index-only scan of 343 leaf pages, `Heap Fetches: 0`, the join unchanged: 3.0 s.
First (iii): two commands, no rewrite, it fixes the node that owns the time. Then (iv) if the remaining 0.4 s scan matters. (i) and (ii) spend effort on the 12 % of the time the scan owns.

Warning / Gotcha: The traps

Reading inner-side `rows=1` as one row (it is per loop). Reading `actual time=a..b` as a duration. "`read` means disk". Judging by the global hit ratio. Fixing the scan when the join owns the time. Blaming the top node. Building the hash on the larger unfiltered table. Adding an index for a 4× estimate error.